

数据访问 DB

目录

一、 概述.....	4
二、 接口说明.....	4
简介.....	4
query 方法.....	5
queryDataSet.....	6
update.....	8
execute.....	8
executeBatch.....	9
timeout.....	10
三、 注意事项.....	11
ResultSetHandler.....	11
优化 in 参数转临时表.....	11
bigint 类型值.....	13
boolean 类型参数.....	13
禁止新建线程访问.....	14
开发环境的 SQL 日志.....	14
四、 案例演示.....	14
注册监听及查询数据.....	15
查询结果集.....	16
删除数据.....	16
更新数据.....	17

执行操作	17
批量执行	18
五、 操作实践	18

一、概述

数据库访问是平台提供的访问数据库的接口 DB。它的基本用法与 JDBC 相似，但是它提供更加统一方便的接口。在使用 DB 访问数据库的时候统一使用 KSQL 语法。在苍穹系统中，支持每个应用使用独立的数据库，SQL 在哪个数据库上执行是通过 DBRoute 来路由。DB 屏蔽了物理数据库信息，用 DBRoute 对应具体的物理库，由 mc 的数据中心管理进行配置。DBRoute 内置了常用的 routeKey，也可以通过 DBRoute.of(“routeKey”)来构造。表单的 routeKey 在设计时已自动指定，在同一个应用下的表单 routeKey 相同。

在需要通过 SQL 访问的场景下必须使用 DB 来访问数据库。

二、接口说明

简介

方法	说明
query	查询
queryDataSet	查询（返回 DataSet）
update	插入、更新或删除
execute	执行任何类型的 SQL
executeBatch	批量执行任何类型的 SQL
timeout	设置超时时间
getDbType	获取数据库类型
existsTable	判断表是否存在
existsIndex	判断表索引是否存在

getPrimaryKeys	获取表主键
getFieldInfo	获取表的字段信息
getIndexInfo	获取表的索引信息
isXDBEnable	是否启用了 xdb 引擎(水平分表)
isSplittingReadEnable	是否开启读写分离
isSplittingAutoReadWrite	是否开启自动读写模式
getWrittenRouteKey	获取当前事务已写的库路由标识

数据库访问 DB 相关接口定义和实现存在于 bos-dbengine-1.0.jar 中。对于使用者来说只需要调用 kd.bos.db.DB 类中的方法即可。

query 方法

查询数据库操作

```
public static <T> T query(DBRoute dbRoute, String sql,
    ResultSetHandler<T> rh)
```

```
public static <T> T query(DBRoute dbRoute, String sql, Object[] params,
    ResultSetHandler<T> rh)
```

返回值：返回 ResultSet 处理器返回的结果

参数	类型	说明
dbRoute	DBRoute	数据库路由
sql	String	查询 SQL 语句
params	Object[]	SQL 参数

rh	ResultSetHandler<T>	jdbc 结果集 ResultSet 处理器
----	---------------------	------------------------

query 示例代码

```
// 不带参数
String sql = "SELECT * FROM table;"
DB.query(DBRoute.basedata, sql, rs -> {
    return rs.next();
});

// 带参数
String sql = "SELECT * FROM table WHERE fid=?;"
DB.query(DBRoute.basedata, sql, new Object[]{"xxxxxxx"}, rs -> {
    return rs.next();
});
```

```
// 查询数据
if (evt.getItemKey().equals("wyjx_selectbtn")) {
    System.out.println("您点击了查询按钮");
    String sql = "SELECT fnumber,fname,fstatus FROM tk_wyjx_emptype";
    Boolean flag = DB.query(DBRoute.basedata, sql, rs -> {
        return rs.next();
    });
    System.out.println(flag);
}
```

queryDataSet

功能描述

查询数据库操作（返回 DataSet）。

方法

```
public static DataSet queryDataSet(String algoKey, DBRoute dbRoute,
String sql)
```

```
public static DataSet queryDataSet(String algoKey, DBRoute dbRoute,
```

String sql, Object[] params)

参数 algoKey 用于跟踪性能，当前执行的时间将被记录到指标系统。这个值，
应能有效定位到代码，如=类名+方法，至少含类名

返回 DataSet 数据集（它是根据查询结果 ResultSet 包装而来）

参数	类型	说明
algoKey	String	algo 标识
dbRoute	DBRoute	数据库路由
sql	String	查询 SQL 语句
params	Object[]	SQL 参数

queryDataSet 示例

```
// 不带参数
String sql = "SELECT * FROM table;";
DataSet dataSet = DB.queryDataSet(this.getClass.getName(), DBRoute.basedata, sql);
// 带参数
String sql = "SELECT * FROM table WHERE fid=?;";
DataSet dataSet = DB.queryDataSet(this.getClass.getName(), DBRoute.basedata, sql, new
Object[]{"xxxxxxx"});
```

```
// 查询结果集
if (evt.getItemKey().equals("wyjx_selectdatasetbtn")) {
    System.out.println("您点击了wyjx_selectdatasetbtn按钮");
    String sql = "SELECT fnumber,fname,fstatus FROM tk_wyjx_emptytype";
    DataSet ds = DB.queryDataSet(this.getClass().getName(), DBRoute.basedata, sql);
    while (ds.hasNext()) {
        Row row = ds.next();
        String number = (String) row.get("fnumber");
        String name = (String) row.get("fname");
        String status = (String) row.get("fstatus");
        System.out.println(number + ":" + name + ":" + status);
    }
}
```

update

功能描述：插入、更新或删除数据库操作

```
public static int update(DBRoute dbRoute, String sql)
```

```
public static int update(DBRoute dbRoute, String sql, Object[] params)
```

返回值：更新数据的行数

参数	类型	说明
dbRoute	DBRoute	数据库路由
sql	String	查询 SQL 语句
params	Object[]	SQL 参数

update 示例

```
// 不带参数
String sql = "UPDATE t_xxx SET field=xxx WHERE fid=xxx;"
DB.update(DBRoute.basedata, sql);
// 带参数
String sql = "UPDATE t_xxx SET field=? WHERE fid=?;"
DB.update(DBRoute.basedata, sql, new Object[]{"xxx", "xxx"});
```

```
String sql = "update tk_wyjx_emptytype set fstatus=? where fstatus=?";
DB.update(DBRoute.basedata, sql, new Object[] { "A", "C" });
```

execute

功能描述：执行任何类型的 SQL

```
public static boolean execute(DBRoute dbRoute, String sql)
```

```
public static boolean execute(DBRoute dbRoute, String sql, Object[]
params)
```


返回值：是否执行成功

参数	类型	说明
dbRoute	DBRoute	数据库路由
sql	String	查询 SQL 语句
params	Object[]	SQL 参数

execute 示例

```
// 不带参数
String sql = "UPDATE t_xxx SET field=xxx WHERE fid=xxx;"
DB.execute(DBRoute.basedata, sql);

// 带参数
String sql = "UPDATE t_xxx SET field=? WHERE fid=?;"
DB.execute(DBRoute.basedata, sql, new Object[]{"xxx", "xxx"});
```

```
String sql = "update tk_wyjx_emptytype set fstatus=? where fstatus=?";
boolean flag = DB.execute(DBRoute.basedata, sql, new Object[] { "C", "A" });
System.out.println("執行結果: " + flag);
```

executeBatch

功能描述：批量执行任何类型的 SQL

```
public static boolean executeBatch(DBRoute dbRoute, String sql,
List<Object[]> paramsList)
```

返回值：批量执行的数量

参数	类型	说明
dbRoute	DBRoute	数据库路由
sql	String	查询 SQL 语句
paramsList	List<Object[]>	SQL 参数列表

executeBatch 示例

```
String sql = "UPDATE t_xxx SET field=? WHERE fid=?";
List<Object[]> paramsList = new ArrayList();
Object[] params = new Object[]{"xxx", "xxx"};
paramsList.add(params);
int[] executeBatch = DB.executeBatch(DBRoute.basedata, sql, paramsList);
```

```
String sql = "update tk_wyjx_emptype set fstatus=? where fstatus=?";
List<Object[]> paramsList = new ArrayList<Object[]>();
paramsList.add(new Object[] { "A", "C" });
int[] results = DB.executeBatch(DBRoute.basedata, sql, paramsList);
System.out.println("執行結果: " + results);
```

timeout

功能描述：设置查询超时时间

```
public static QueryTimeout timeout(int seconds)
```

返回值：查询超时对象

默认情况下，执行 SQL 超时时间为 300 秒（可通过在 MC 配置 db.query.timeout=300 来修改）。timeout 方法允许开发者自定义查询超时时间。执行超过 query time，则执行中断并抛出 SQLException

示例代码：

```
try(DB.timeout(10)) {
    DB.query(...);
}
```

}

参数	类型	说明
seconds	int	超时时间（秒）

三、注意事项

ResultSetHandler

```
public interface ResultSetHandler<T> {  
  
    T handle(ResultSet rs) throws Exception;  
  
}
```

ResultSetHandler 是用于处理 JDBC 结果集 ResultSet 的一个自定义的处理器接口。使用者可以实现该接口方法来实现结果集的处理逻辑, 实现的方法中的返回值也是 DB.query 方法的返回值。

优化 in 参数转临时表

在数据库操作中, 由于数据库对 in 参数有长度限制, 所以我们需要对 in 参数转临时表处理。

当启用 in 优化, in 的参数个数 >= 优化阈值时, 将对过滤条件信息优化。in 参数个数超过 orm.opt.in.maxsize, 则系统抛异常, 异常信息形如

in query params length over the maximum: 510000 > 500000

in 优化参数配置

优化阈值：默认 1000

orm.opt.in.threshold=1000

in 允许的最大个数：默认 50w

orm.opt.in.maxsize=500000

为了优化 in 参数转临时表，在 DB 中提供了专门的 in 参数转临时表数据库操作接口。

```
public static boolean execute(DBRoute dbRoute, SqlBuilder sb)
```

```
public static <T> T query(DBRoute dbRoute, SqlBuilder sb,  
ResultSetHandler<T> rh)
```

```
public static DataSet queryDataSet(String algoKey, DBRoute dbRoute,  
SqlBuilder sb)
```

这些接口与其他数据库操作接口使用并无太大差别。在有需要 in 参数转临时表时，使用上述三个接口，只是这些接口需要使用 SqlBuilder 来创建 SQL 语句

示例代码

```
public void testDBQuerySqlBuilder(){  
    //设置启用阈值  
    System.setProperty("orm.opt.in.threshold", "100");  
    List<Object> pkIds = getPKs(100);  
    //original sql= SELECT FID FROM T_SEC_USER WHERE FID IN(?,?,?... ) ORDER BY FID DESC  
    SqlBuilder sqlBuilder = new SqlBuilder();  
    sqlBuilder.append("SELECT FID FROM T_SEC_USER WHERE");  
    sqlBuilder.appendIn("FID", pkIds);  
    sqlBuilder.append("ORDER BY FID DESC");  
    DataSet ds = DB.queryDataSet("test_sqlBuilder", DBRoute.basedata, sqlBuilder);  
}
```

示例代码

```
public List<Object> getPKs(int size){
    String sql = "SELECT TOP " + size + ",0 FID FROM T_SEC_USER";
    DataSet ds = DB.queryDataSet("InQueryToTempTable_getPKs", DBRoute.base, sql);
    List<Object> list = new ArrayList<>();
    for(Row r:ds){
        Object o = r.get(0);
        list.add(o);
    }
    return list;
}
```

空格和 0 长度串

数据库字段若为字符串类型，则不允许保存 null 和 0 长度的字符串，无值默认用" "。

用户仍正常使用，存取值由 DB 层处理转换：

保存：参数传入" "或 null，则保存为" "。 必须通过参数设置才会自动转换。

(ParameterSetter)

查询：" "转为" "。 通过 DB 结果集转换。(WrapCloseResultSet)

控制是否转换的参数：db.spaceAsEmptyString，默认值为 true。

bigint 类型值

ksql bigint 类型的字段，取值返回类型统一 为 long

boolean 类型参数

参数传 boolean 类型，则在 jdbc 参数设置时自动转为' 0' 或' 1'。(对应的存储字段类型

应为 char(1))

注意：在 ResultSet 中取值仍为' 0' 或' 1'，在 ORM 的 DynamicObject 中的值则为 true

或 false。

禁止新建线程访问

若需要开启新线程，用 ThreadPools 启动。自行 new Thread 被禁止使用 DB 访问数据库。

开发环境的 SQL 日志

在开发环境，如果 info 级别的日志被关闭，则可以通过以下代码，在控制台输出。

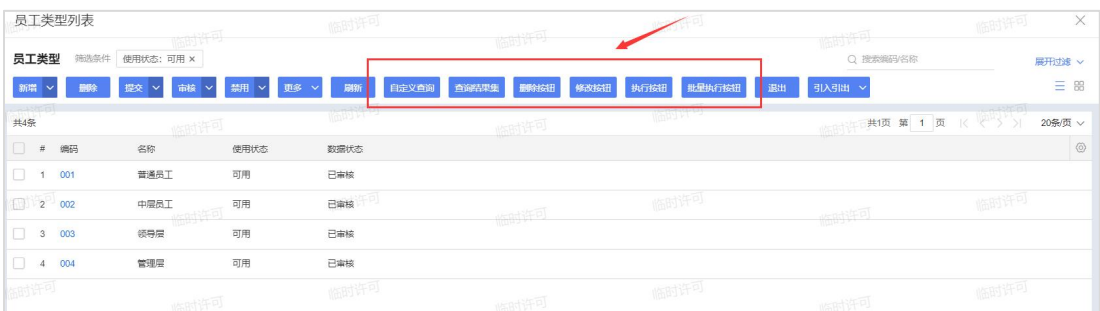
```
// 启动前开启 DB 日志（加在启动类里）
```

```
System.setProperty("db.sql.out", "true");
```

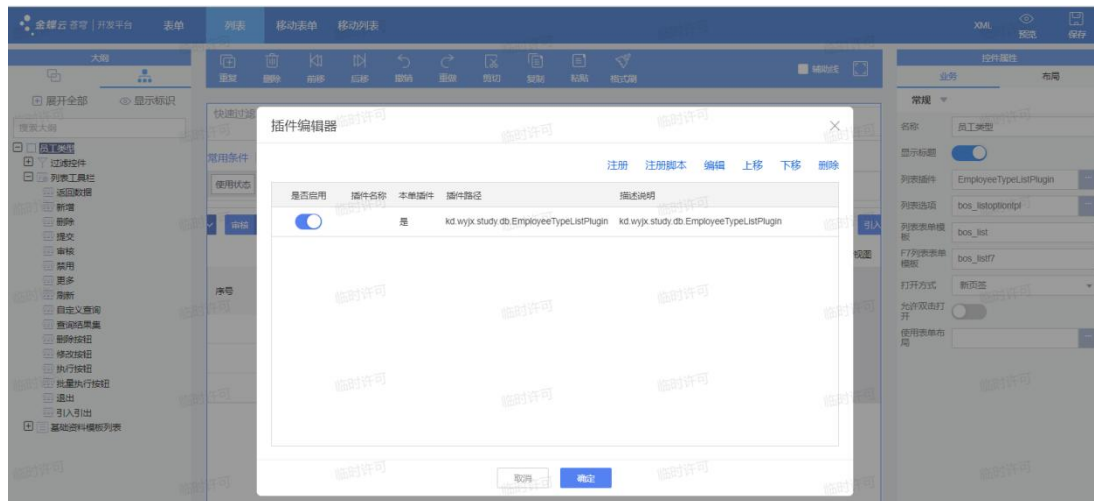
```
// 运行期前开启 DB 日志(仅用于调试，不要在产品中使用，会导致日志配置失效)
```

```
DB.setSqlOut(true);
```

四、案例演示



#	编码	名称	使用状态	数据状态
1	001	普通员工	可用	已审核
2	002	中层员工	可用	已审核
3	003	领导层	可用	已审核
4	004	管理层	可用	已审核



注册监听及查询数据

```
public class EmployeeTypeListPlugin extends AbstractListPlugin {  
    @Override  
    public void registerListener(EventObject e) {  
        super.registerListener(e);  
        this.addClickListeners("wyjx_selectbtn");  
        this.addClickListeners("wyjx_selectdatasetbtn");  
        this.addClickListeners("wyjx_deletebtn");  
        this.addClickListeners(new String[] { "wyjx_updatebtn",  
        "wyjx_executebtn" ,"wyjx_batchexecutebtn"});  
    }  
    @Override  
    public void itemClick(ItemClickEvent evt) {  
        super.itemClick(evt);  
        // 查询数据  
        if (evt.getItemKey().equals("wyjx_selectbtn")) {  
            System.out.println("您点击了查询按钮");  
            String sql = "SELECT fnumber,fname,fstatus FROM tk_wyjx_emptype";  
            Boolean flag = DB.query(DBRoute.basedata, sql, rs -> {  
                return rs.next();  
            });  
            System.out.println(flag);  
        }  
    }  
}
```

查询结果集

```
// 查询结果集
if (evt.getItemKey().equals("wyjx_selectdatasetbtn")) {
    System.out.println("您点击了 wyjx_selectdatasetbtn 按钮");
    String sql = "SELECT fnumber,fname,fstatus FROM tk_wyjx_emptytype";
    DataSet ds = DB.queryDataSet(this.getClass().getName(), DBRoute.basedata,
sql);

    while (ds.hasNext()) {
        Row row = ds.next();
        String number = (String) row.get("fnumber");
        String name = (String) row.get("fname");
        String status = (String) row.get("fstatus");
        System.out.println(number + ":" + name + ":" + status);
    }
}
```

删除数据

```
// 删除数据
if (evt.getItemKey().equals("wyjx_deletebtn")) {
    System.out.println("删除按钮");
    String sql = "delete from tk_wyjx_emptytype where fstatus=?";
    boolean flag = DB.execute(DBRoute.basedata, sql, new Object[] { "A" });
    System.out.println("删除数据成功");
}
```


更新数据

```
// 更新数据
if (evt.getItemKey().equals("wyjx_updatebtn")) {
    // 不带参数
    // String sql = "UPDATE t_XXX SET field=xxx WHERE fid=xxx;"
    // DB.update(DBRoute.basedata, sql);
    // 带参数
    // String sql = "UPDATE t_XXX SET field=? WHERE fid=?;"
    // DB.update(DBRoute.basedata, sql, new Object[]{"xxx", "xxx"});
    String sql = "update tk_wyjx_emptytype set fstatus=? where fstatus=?";
    DB.update(DBRoute.basedata, sql, new Object[] { "A", "C" });
}
```

执行操作

```
// 执行操作
if (evt.getItemKey().equals("wyjx_executebtn")) {
    // public static boolean execute(DBRoute dbRoute, String sql)
    // public static boolean execute(DBRoute dbRoute, String sql, Object[]
params)

    String sql = "update tk_wyjx_emptytype set fstatus=? where fstatus=?";
    boolean flag = DB.execute(DBRoute.basedata, sql, new Object[] { "C", "A" });
    System.out.println("執行結果: " + flag);
}
```

批量执行

```
// 批量执行
if (evt.getItemKey().equals("wyjx_batchexecutebtn")) {
    // String sql = "UPDATE t_xxx SET field=? WHERE fid=?";
    // List<Object[]> paramsList = new ArrayList();
    // Object[] params = new Object[]{"xxx", "xxx"};
    // paramsList.add(params);
    // int[] executeBatch = DB.executeBatch(DBRoute.basedata, sql, paramsList);
    String sql = "update tk_wyjx_emptytype set fstatus=? where fstatus=?";
    List<Object[]> paramsList = new ArrayList<Object[]>();
    paramsList.add(new Object[] { "A", "C" });
    int[] results = DB.executeBatch(DBRoute.basedata, sql, paramsList);
    System.out.println("執行結果: " + results);
}
```

五、操作实践

请完成以下需求场景的开发：

办公用品登记单入库操作时先查询办公用品库存单是否有该物品的库存记录, 如果有则办公用品库存单对应的物品库存数量增加即可, 如果没有对应的库存记录, 则需要增加一条办公用品库存记录, 库存数量等于办公用品登记单的数量。